

Specification and Verification of Embedded and Cyber-Physical Systems

Exercises — ESBMC 8.5

Before you start

Every command below was run against ESBMC **8.5.0**, and the sources are in `exercises/ecps/`. The LTL exercises additionally need `libltl2ba`, the ESBMC fork of `ltl2ba`; the upstream `ltl2ba` shipped by Homebrew and most distributions has no `-O c` backend and cannot produce an ESBMC monitor.

```
esbmc --version                # expect 8.5.0 or newer
git clone https://github.com/esbmc/libltl2ba.git && make -C libltl2ba
cd exercises/ecps && ./run-all.sh    # every verdict below
```

The one thing that catches everybody. ESBMC does *not* negate your LTL formula for you. The polarity you hand `ltl2ba` is the question you are asking:

<code>-f '!G(...)'</code>	does the property <i>hold</i> here?	<code>LTL_SUCCEEDING</code> $\Rightarrow$ <code>SUCCESSFUL</code>
<code>-f 'G(...)'</code>	can it be <i>refuted</i> here?	<code>LTL_FAILING</code> $\Rightarrow$ <code>FAILED</code>

Use the negated form when you are asking whether a controller is correct. When you report a verdict, say which polarity produced it.

1 Bounded model checking

1) (Bounded Model Checking)

Describe the steps involved in checking a program with the BMC technique, from reading the program to generating the SMT formulas. Use the actuator ramp below to describe each step: the heater duty cycle rises by a fixed increment on every tick of the control loop. `__ESBMC_assume` constrains (non-deterministic) variables and `assert` states the property to check.

```
/* exercises/ecps/ramp.c */
#include <assert.h>
#define STEP 2                /* duty increment per tick */

int main(int argc, char **argv) {
    long long int tick = 1, duty = 0;
    unsigned int ticks = 5;    /* ticks in this ramp */
    __ESBMC_assume(ticks >= 1);
    while (tick <= ticks) {
        duty = duty + STEP;
        tick++;
    }
    assert(duty == ticks * STEP);
}
```

- Unroll the loop by hand for `ticks = 5` and write the resulting single-assignment (SSA) form.
- Split the SSA form into the constraint C and the property P , and give the formula the solver is asked to decide. Which answer — *satisfiable* or *unsatisfiable* — corresponds to `VERIFICATION FAILED`?
- Run `esbmc ramp.c`. Compare the assignment count reported by *Symex completed* with the number surviving *Slicing*. What did the slicer remove, and why is removing it sound?

- (d) Replace `ticks = 5` with `ticks = nondet_uint()` and run `esbmc ramp.c --unwind 6`. Explain what `--unwind` now controls and what a failing *unwinding assertion* is telling you.
- (e) The same control loop in Python is `exercises/ecps/settle.py`: the heater ramps in fixed 2-degree steps until the measured temperature equals the setpoint.

```
def settle(setpoint: int) -> int:
    temp = 0
    while temp != setpoint:
        temp = temp + 2
    return temp

settle(10) # reachable in 5 ticks
settle(7)  # unreachable: an odd setpoint is never hit in steps of 2
```

Run `esbmc settle.py --unwind 8`. Explain what a bounded model checker can and cannot tell you about termination here, and state the precondition on `setpoint` that makes the loop terminate.

2 Concurrency and mutual exclusion

2) (Race condition on the display bus)

The T and P tasks hand their samples to the S task over a single-word display bus, and only one task may drive the bus at a time. `ecps/bus.c` arbitrates with Peterson's algorithm; `in_T` and `in_P` record which task currently owns the bus.

```
/* exercises/ecps/bus.c */
void *task_T(void *arg) {
    want[0] = 1;
    turn = 1;
    while (want[1] && turn == 1) {}
    in_T = 1;
    bus_word = 1; /* temperature */
    in_T = 0;
    want[0] = 0;
    return 0;
}

void *task_P(void *arg) {
    want[1] = 1;
    turn = 0;
    while (want[0] && turn == 0) {}
    in_P = 1;
    bus_word = 2; /* pressure */
    in_P = 0;
    want[1] = 0;
    return 0;
}
```

- (a) Specify the following three properties in LTL, using c_i for “task i is driving the bus” (`in_T`, `in_P`) and r_i for “task i has requested the bus” (`want[0]`, `want[1]`).
- At most one task is driving the bus at any time.
 - Whenever a task requests the bus, it eventually gets it.
 - A task can eventually request the bus.
- (b) Which are *safety* properties and which are *liveness* properties? Which can a bounded run refute, and which can it only fail to refute?
- (c) Verify property (i) with ESBMC. Generate the monitor and run it:

```
ltl2ba -O c -H '"bus.h"' -f '!G(!({in_T} && {in_P}))' > bus-mutex-neg.ba-2.c
esbmc bus.c --ltl bus-mutex-neg.ba-2.c -DLTL_PREFIX_BOUND=10 --context-bound 3
```

Report the *Final lowest outcome*.

- (d) Now break the algorithm: delete `turn = 1;` from `task_T` and re-run the same monitor. The outcome changes to `LTL_BAD`, not `LTL_FAILING`. Explain the difference between the two, and give the interleaving that puts both tasks on the bus at once.

- (e) Encode property (ii) and check it. A bounded run cannot establish it — say what verdict you get and why that is the expected answer rather than a tool limitation.

3 Specifying and verifying an embedded system

3) (Cyber-threats and control properties)

A typical embedded system consists of a *human-machine interface* (keyboard and display), a *processing unit* (real-time computer system), and an *instrumentation interface* (sensor, network, and actuator) that can be connected to some physical plant. Figure 1 illustrates a simple embedded system employed in a chemical process, which can be operated remotely via TCP/IP. It contains two Analog-to-Digital Converters (ADC) and a Digital-to-Analog Converter (DAC). The first ADC reads the thermocouples' temperature; the second reads the pressure transducer. The DAC sets the desired position of the valve/pump, and a switch turns the heater on or off. The overall objective is to keep the temperature and pressure of the chemical process within well-defined limits, under a remote operator.

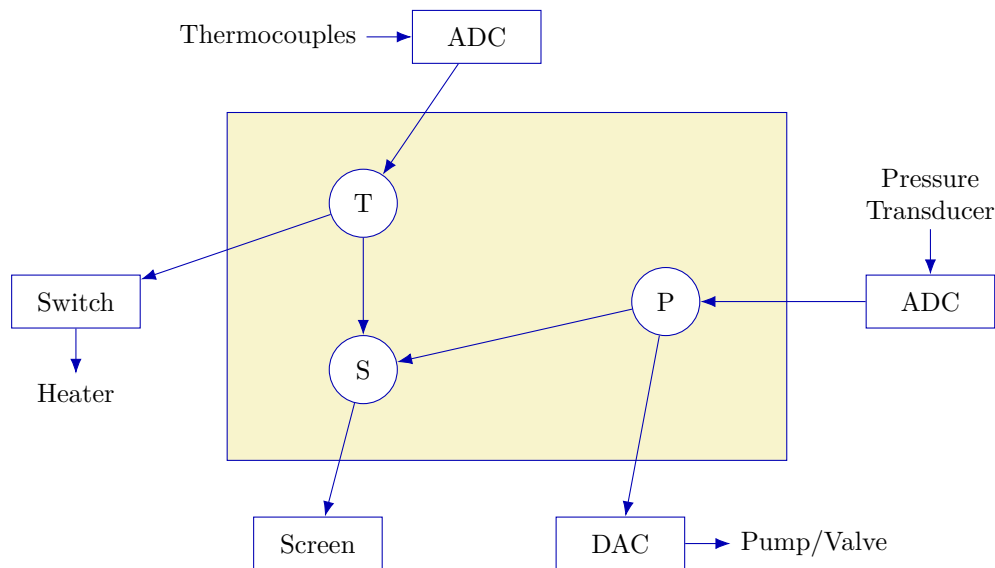


Figure 1: Simple embedded system [7].

`exercises/ecps/plant.c` implements this system as a cyclic executive: each round runs T, P and then S. The two ADC readings are non-deterministic within their transducer ranges, so one ESBMC run covers every reading the hardware can produce.

```

/* exercises/ecps/plant.c -- abridged */
static void process_T(void) { /* temperature control */
    temperature = read_thermocouple(); /* nondet, -50..400 degC */
    if (temperature < TEMP_LOW) heater_on = 1;
    else if (temperature > TEMP_HIGH) heater_on = 0;
}
static void process_P(void) { /* pressure control */
    pressure = read_transducer(); /* nondet, 0..600 bar */
    if (pressure > PRESS_HIGH) valve_open = 1;
    else if (pressure < PRESS_LOW) valve_open = 0;
}
static void process_S(void) { /* screen refresh */
    if (fresh_sample && !bus_busy()) { displayed = 1; fresh_sample = 0; }
}

```

- (a) Specify the following properties of this embedded system using LTL. The propositions available to the monitor are declared in `plant.h`: `temperature`, `pressure`, `heater_on`, `valve_open`, `fresh_sample` and `displayed`.
- A T process reads the measured values from a temperature sensor, turns the heating system on if the temperature is below 20 °C, and turns off the heating system if the temperature is above 300 °C.
 - Process P regulates pressure with a sensor opening the pump/valve when the pressure value is above 500 bar and closing the pump/valve when the pressure value is below 100 bar.
 - The measured values will be shown on a liquid crystal display (LCD) when the T and P processes transfer data to an S process.
- (b) Verify each of your three formulas with ESBMC. Generate the monitor from the *negated* formula, since you are asking whether the property holds:

```
ltl2ba -O c -H '"plant.h"' -f '!G({pressure > 500} -> F {valve_open})' \
    > valve-neg.ba-2.c
esbmc plant.c --ltl valve-neg.ba-2.c -DLTL_PREFIX_BOUND=10
```

Exactly one of the three reports VERIFICATION SUCCESSFUL. Report all three outcomes.

- The heater property fails. Read the counterexample and find the defect. *Hint*: look at the value of `temperature` before the first round runs, and at what the specification therefore requires of the controller at that instant. Why does the pressure property not have the same problem?
- Repair `plant.c` so the heater property holds, re-verify, and state what your repair assumes about the hardware.
- The screen property also fails, for an entirely different reason. Diagnose it, and say what assumption about the display bus you would have to add — and how you would express it — for the property to become provable.
- What are the **security objectives** of this simple embedded system?
- What are the sources of security problems that can arise when this system is operated remotely? Consider the TCP interface specifically, and say which of your LTL properties would still hold if an attacker could set `heater_on` directly.

References

- [1] L. C. Cordeiro, B. Fischer, J. Marques-Silva. “SMT-Based Bounded Model Checking for Embedded ANSI-C Software”. *IEEE Transactions on Software Engineering* 38(4):957–974, 2012. doi:10.1109/TSE.2011.59
- [2] M. Y. R. Gadelha, H. I. Ismail, L. C. Cordeiro. “Handling loops in bounded model checking of C programs via k-induction”. *International Journal on Software Tools for Technology Transfer* 19(1):97–114, 2017. doi:10.1007/s10009-015-0407-9
- [3] M. Y. R. Gadelha, F. R. Monteiro, J. Morse, L. C. Cordeiro, B. Fischer, D. A. Nicole. “ESBMC 5.0: an industrial-strength C model checker”. *ASE 2018*, pp. 888–891. doi:10.1145/3238147.3240481
- [4] B. Farias, R. Menezes, E. B. de Lima Filho, Y. Sun, L. C. Cordeiro. “ESBMC-Python: A Bounded Model Checker for Python Programs”. *ISSTA 2024*, pp. 1836–1840. doi:10.1145/3650212.3685304

- [5] N. Tihanyi, Y. Charalambous, R. Jain, M. A. Ferrag, L. C. Cordeiro. “A New Era in Software Security: Towards Self-Healing Software via Large Language Models and Formal Verification”. *AST@ICSE 2025*, pp. 136–147. doi:10.1109/AST66626.2025.00020
- [6] C. Baier, J.-P. Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN 978-0-262-02649-9.
- [7] A. Burns, A. J. Wellings. *Real-Time Systems and Programming Languages: Ada, Real-Time Java and C/Real-Time POSIX*, 4th edition. Addison-Wesley, 2009. ISBN 978-0-321-41745-9. Source of the chemical-process case study in Figure 1.